

Field guide

Verifying a neural network, end to end — the triage exercise on paper

What this is. Everything the Colab notebook does, with every command and its real output, so that you can follow along without a laptop, or check what you see against what you should see. Vehicle is the specification language used; the subject is *verification of neural networks*, and the properties export to the tool-neutral VNN-LIB format.

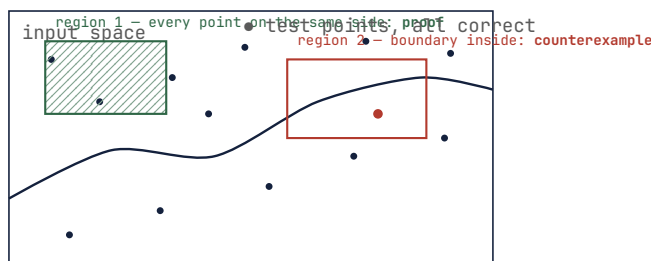
Everything is synthetic and educational. The triage example borrows its labelling rule from the NHS National Early Warning Score 2 (Royal College of Physicians, 2017; Scale 1 only, no age or pregnancy adjustments) so that the properties feel real. The data are generated, the network is a toy, and nothing here is a clinical tool.

Why verify

A test set is a handful of points. A deployed system meets inputs nobody sampled. Three recent examples, all from systems that had passed their tests:

- **20 April 2026** — a Waymo robotaxi drove into a flooded road in San Antonio and was swept into a creek; 3,791 cars were recalled. The software had been written to *slow, not stop* at water it could not judge: the perception worked, the *specification* was wrong (CNBC, 12 May 2026).
- **21 April 2026** — the High Court upheld the Metropolitan Police’s live facial recognition after Shaun Thompson, wrongly matched, was detained; the Met scanned 4.2 million faces in 2025 with ten false alerts, eight of them Black people (The Register; PA).
- **February 2026** — AI-enabled devices were reported to account for nearly half of FDA medical-device recalls, mostly for diagnostic or measurement error (*Biomedical Safety & Standards*).

Testing tells you about the inputs you tried. **Verification** tells you about the inputs you did not: a verifier either *proves* that a property holds for **every** input in a region, or returns a concrete **counterexample**.



Vocabulary

Property / specification

A statement of the form “for every input x in region R , the network’s output satisfies $\varphi(x)$ ”, written in the units of the problem.

Region

The set of inputs the statement covers: a box of clinical values, an ε -ball around an image, “every geometry with the intruder dead ahead”. Choosing it is a design decision, not a solver setting.

Counterexample

A concrete input in the region that violates the property. Verifiers report it in the specification's units when the specification is written that way.

Sound / complete

A sound verifier never claims a property holds when it does not. A complete one always answers (given time). Marabou is both; its answers are proof, counterexample, or timeout.

Robustness, ϵ -ball

The property “every input within ϵ of this one gets the same class”. ϵ is a *parameter*; someone has to choose it.

Verifier

The solver that answers the question. Marabou (Reluplex's successor) case-splits ReLU networks into linear problems; other tools use branch-and-bound or abstract interpretation. Verification is NP-complete: small networks take seconds, large ones minutes to never.

ONNX / VNN-LIB

The two standard formats — for networks and for properties — that every tool in the annual VNN-COMP competition reads (2025: eight tools).

Embedding gap

Requirements live in feet, mmHg and advisories; solvers live in normalised tensors. The translation is where specifications quietly go wrong; a specification *language* exists to close it.

The toolchain

Vehicle (`pip install vehicle-lang`) is a typed specification language: you write properties in the problem's units, supply the network (ONNX) and any datasets or parameters on the command line, and it compiles the property to solver queries, to training losses, or to a theorem prover. **Marabou** (`pip install maraboupy`) is the solver. Marabou ships wheels only for Python 3.8–3.11 on x86_64, so the Colab notebook creates a separate Python 3.11 environment with `environment/colab_bootstrap.sh` (two to three minutes) and calls the `vehicle` command from there; nothing is imported into the notebook's own kernel.

```
$ vehicle typecheck -s specs/triage.vcl
$ vehicle verify -s specs/triage.vcl -n triage:models/triage-v1.onnx -y hypoxiaNeverLow \
  --solver Marabou -a "--timeout=30"
$ vehicle compile queries --format VNNLibQueries -o out/ -s specs/triage.vcl \
  -n triage:models/triage-v3.onnx -e hypoxiaNeverLow
```

the three commands you will use

The triage exercise, step by step

The system

Seven inputs in clinical units — respiration rate (breaths/min), SpO₂ (%), systolic blood pressure (mmHg), pulse (beats/min), temperature (°C), on supplemental oxygen (0/1), not alert (0/1) — and three output scores: *low*, *medium*, *high* urgency. The network advises the class with the highest score. Three networks (7→16→16→3, ReLU) were trained on the same 40 000 synthetic patients:

model	training labels	acc. on own labels	acc. on the strict NEWS2 bands
triage-v1	a rule that <i>forgot</i> “3 points in any single parameter is a trigger”	95.6 %	87.5 %
triage-v2	the correct NEWS2 bands, ordinary training	95.9 %	95.9 %
triage-v3	correct bands + the properties as training losses + a one-unit safety margin	95.6 %	91.9 %

The specification (specs/triage.vcl, excerpt)

triage.vcl

```

type Patient = Tensor Real [7]          -- rr, spo2, sbp, pulse, temp, o2, cvpu (clinical units)
type Scores  = Tensor Real [3]          -- low, medium, high

@network
triage : Patient -> Scores

validPatient : Patient -> Bool           -- the REGION: every plausible patient
validPatient x = 4 ≤ x ! rr ≤ 60 and 50 ≤ x ! spo2 ≤ 100 and 40 ≤ x ! sbp ≤ 250 and
                20 ≤ x ! pulse ≤ 200 and 30 ≤ x ! temp ≤ 43 and 0 ≤ x ! o2 ≤ 1 and 0 ≤ x !
                cvpu ≤ 1

alertOnAir : Patient -> Bool             -- flags pinned with = (verifiers only know real numbers)
alertOnAir x = x ! o2 = 0 and x ! cvpu = 0

advises : Patient -> Index 3 -> Bool    -- "class c has the strictly highest score"
advises x c = forall j . j ≠ c => triage x ! c > triage x ! j

@property
hypoxiaNeverLow : Bool                  -- SpO2 ≤ 91 scores 3 points: never low urgency
hypoxiaNeverLow = forall x . validPatient x and alertOnAir x and x ! spo2 ≤ 91 => not (advises x
low)

```

The other hard limits follow the same shape: shockNeverLow ($SBP \leq 90$), notAlertNeverLow, hypoxiaOnOxygenNeverLow, and normalVitalsAlwaysLow (all vitals comfortably in the 0-point ranges $\Rightarrow$ low; over-triage matters too). noiseRobust quantifies over a perturbation d around each of 12 evaluation patients.

Step 1 – model v1, first property

```
$ vehicle verify -s specs/triage.vcl -n triage:models/triage-v1.onnx -y hypoxiaNeverLow --solver
Marabou
```

EXPECTED OUTPUT

```

hypoxiaNeverLow [=====] 1/1 queries
result: ✗ - Marabou found a counterexample
x: [ 14.26236, 86.346347, 99.147873, 66.652075, 36.402779, 0.0, 0.0 ] (0.5 s)

```

Reading the counterexample. Respiration 14, SpO₂ 86 %, SBP 99, pulse 67, 36.4 °C, alert, on air. NEWS2 points: 0, 3, 1, 0, 0, 0, 0 $\rightarrow$ aggregate 4, highest single parameter 3 $\rightarrow$ the rule says **medium**. triage-v1 advises **low**. The model is 96 % accurate on the labels it was given; the labels encoded the wrong rule. **COUNTEREXAMPLE**

Step 2 – model v2

hypoxiaNeverLow on triage-v2

```
result: ✓ - Marabou proved no counterexample exists (12.5 s)
```

result: ✗ - Marabou found a counterexample

x: [13.174745, 93.023361, 90.0, 55.68502, 36.623059, 0.0, 0.0]

shockNeverLow on triage-v2

(0.4 s)

SBP 90.0 mmHg **exactly**; everything else unremarkable. Evaluate the network there and the *low* and *medium* scores are equal to within rounding: the learned decision boundary passes through the edge of the region. “Advises medium” requires a strict winner, so a tie is a failure — a triage system that cannot decide is not safe. Charted readings are integers, so no test on real charts would land *at* 90.0; the verifier probes real numbers. `notAlertNeverLow` fails the same way.

Step 3 – model v3

```
hypoxiaNeverLow      verified  0.6 s
hypoxiaOnOxygenNeverLow  verified  0.8 s
shockNeverLow        verified  0.9 s
notAlertNeverLow     verified  4.4 s
normalVitalsAlwaysLow verified  0.6 s
```

all five hard-safety properties on triage-v3

VERIFIED Five proofs, each about every plausible patient. The price: v3 agrees with the strict rule on 91.9 % of patients because it deliberately over-triages at the thresholds (SpO₂ 92 %, SBP 91). **Would you deploy v3?** That trade is the responsible-AI decision, and it belongs to people, not to the solver.

Step 4 – robustness to measurement noise

```
$ vehicle verify -s specs/triage.vcl -n triage:models/triage-v3.onnx -y noiseRobust --solver
  Marabou \
  -d patients:data/triage-eval-patients.idx -d labels:data/triage-eval-labels.idx \
  -p epsRR:1 -p epsSpo2:1 -p epsSbp:3 -p epsPulse:2 -p epsTemp:0.1
```

```
noiseRobust:  verified 9/12   falsified 3/12   timed-out 0/12   errored 0/12   (EXPECTED OUTPUT)
                                                         (~3 s)
```

patients provably stable (of 12)	$1 \times \varepsilon$	$2 \times \varepsilon$	$4 \times \varepsilon$
triage-v2	9	8	7
triage-v3	9	9	3

Two of the twelve patients are **boundary** cases (SpO₂ 92 % is one point from a 3-point score): for them a falsification is the *rule* changing inside the box, not a network bug. And note v3 at 4ε : a safety margin on one side is a robustness cost on the other.

Step 5 – edit the specification

`specs/triage-exercises.vcl` appends four exercises (search for TODO): temperature extremes ($\leq 35.0^\circ\text{C}$ scores 3), hypertensive crisis ($\text{SBP} \geq 220$), tighter and looser noise boxes, and the relaxation trap ($0 \leq x \leq 1$ instead of $=$, which yields a “0.37 on oxygen” patient). Typecheck first; the error messages carry line numbers.

Step 6 – not tied to one tool

`vehicle compile queries --format VNNLibQueries -e hypoxiaNeverLow` writes the property as VNN-LIB (`hypoxiaNeverLow-query1.txt`): the network declaration, the negated conclusion as assertions on the outputs, and the input bounds of the region in clinical units. Any VNN-COMP tool can take that file together with the ONNX network.

Design decisions, in plain language

- **Integers vs reals.** NEWS2 is defined on charted integers; the verifier explores real numbers. We define the class of a real-valued reading as the class of its truncated chart value, so the labels change at integers and every specification threshold ($\text{sbp} \leq 90$) sits a full unit from the nearest label change. Rounding instead put the change at 90.5 and asked the network for an infinitely sharp step — unprovable.
- **Conservative labels for v3.** Worst NEWS2 score within ± 1 unit of measurement uncertainty: the model's own boundary lands two units from each threshold, and the proofs go through in seconds. Repairing counterexamples one at a time (a verifier-in-the-loop repair) chased slivers for eight rounds and never converged; the margin is the mechanism.
- **Flags by equality.** Boolean inputs are not allowed; $0 \leq o2 \leq 1$ admits nonsense patients and a disjunction doubles the query count.
- **Small network, normalisation inside.** 16-16 hidden units keep proofs to seconds (32-32 gave 20–30 s and timeouts). Normalisation is folded into the first layer so the specification is in clinical units and the solver only sees Gemm and ReLu.

The stretch: fairness as a property

A loan-approval network takes six numbers including a protected attribute. “Applicants that differ only in the attribute get the same decision” applies the network *twice*, which Marabou cannot express. `models/loan-twin-*.onnx` are two weight-shared copies stacked block-diagonally into one network (input 12 = $A \mathbin{++} B$, output 4); `specs/loan-twin.vcl` then states `fairAtoB` and `fairBtoA` as ordinary properties (idea: Athavale et al., CAV 2024).

```

loan-twin-v1, margin 0.1
fairAtoB  ✖ counterexample  p: [ 10.0, 0.0, 0.17, 0.0, 18.0, 0.0 | 10.0, 0.0, 0.17, 0.0, 18.0, 1.0
]      (1 s)
fairBtoA  ✔ proved no counterexample exists                                (2 s)

```

Identical numbers, protected 0 approved, protected 1 denied. `loan-v2` cannot read the attribute — its first-layer weights on that column are exactly zero, which is a one-line *structural* proof of counterfactual fairness; we do not run the solver on it, because two identical sub-networks side by side make the query degenerate (Marabou loops without honouring its timeout). Yet `v2`'s approval rates by group are still 39 % vs 28 %: income is a proxy. Individual counterfactual fairness is a property; group outcomes are a policy. Credit scoring is high-risk under the EU AI Act (Annex III); the UK Equality Act 2010 covers indirect discrimination.

Troubleshooting

symptom	cause and fix
setup cell takes >4 min	Colab cold start; wait, or Runtime → Restart and re-run (the cell is idempotent)
Marabou: command not found	PATH not set in this kernel — re-run the setup cell
permission deniedvclo	Vehicle writes a compiled file next to the spec; the folder must be writable
Marabou timed out	a real answer: the query was too hard in 30 s. Raise <code>--solver-args "--timeout=N"</code> or shrink the region
wall-clock cap reached	the notebook's helper killed a solver that ignored its own timeout (degenerate queries do this)
counterexample with <code>o2 = 0.37</code>	the relaxation exercise — pin flags with <code>=</code>
mismatch in function declaration names	an untyped constant right after <code>@network</code> ; move constants above the network
syntax error ... before <code>epsRR</code>	<code>@parameter</code> must be on its own line, the declaration on the next
internal error mentioning <code>atVector</code> record	<code>records + @dataset + foreach</code> (Vehicle 0.27.1); use plain tensors
strict-inequality warning	Marabou relaxes <code>></code> to <code>≥</code> ; harmless here, and the reason the fairness spec has a margin

Appendix — installing the toolchain yourself

platform	works?	how
Linux x86_64, Python 3.10–3.11	yes	<code>pip install vehicle-lang==0.27.1 maraboupy==2.0.0 "numpy<2"</code>
Linux x86_64, any Python	yes	<code>bash environment/colab_bootstrap.sh</code> (Python 3.11 venv via <code>uv</code>)
macOS Intel	yes	as Linux
macOS Apple Silicon	not natively	Colab; or an x86_64 Python 3.11 under Rosetta (<code>uv python install cpython-3.11-macos-x86_64-none</code>); or Docker
Windows	not for Marabou wheels	WSL2 or Docker
Docker (any)	yes	<code>docker build --platform linux/amd64 -t nnv-colab environment/</code>

Glossary

advises — the class with the strictly highest score · ε — the size of the perturbation box in a robustness property · **hyperproperty** — a property about two or more evaluations (monotonicity, fairness); needs an encoding such as a twin network · **Marabou** — SMT-based complete verifier for ReLU networks · **NEWS2** — National Early Warning Score 2, RCP 2017 · **ONNX** — the network exchange format · **region** — the set of inputs a property covers · **Vehicle** — the specification language used here · **VNN-LIB** — the property exchange format · **VNN-COMP** — the annual verification competition.